



MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA

UNIVERSIDAD NACIONAL EXPERIMENTAL DE LAS
TELECOMUNICACIONES E INFORMÁTICA

PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

Evaluación Practica #1: Resolución de ejercicios

NodeJS y TypeScript

AUTOR: Jaime Patiño
SECCIÓN: 8-B
DOCENTE; Carlos Márquez
UNIDAD CURRICULAR: Programación III

Caracas, Mayo 2026

Índice

Introducción	2
1. Ejercicio 1: Intercepción de Peticiones y Manejo de Vistas en Node.js	3
1.A Descripción de la Solución	3
1.B Implementación de Código	3
1.C Resultados	5
1.D Explicación de la Lógica	7
2. Ejercicio 2: Uso de Enumeradores en TypeScript	8
2.A Descripción de la Solución	8
2.B Implementación de Código	8
2.C Resultado	9
2.D Explicación de la Lógica	9
Referencias	10

Introducción

El presente documento detalla la implementación técnica de dos ejercicios fundamentales en el desarrollo backend y tipado fuerte. El primer ejercicio aborda la creación de un servidor web utilizando las herramientas nativas de Node.js, prescindiendo de frameworks externos para comprender el ciclo de vida de una petición HTTP (Request) y la generación de una respuesta dinámica (Response) mediante la manipulación de flujos de datos.

El segundo ejercicio se centra en las ventajas de TypeScript para la definición de estructuras constantes y legibles mediante el uso de enumeradores (Enums). Esta práctica permite gestionar listas cerradas de datos, como géneros cinematográficos y países, garantizando la integridad de la información y facilitando el mantenimiento del código fuente.

1. Ejercicio 1: Intercepción de Peticiones y Manejo de Vistas en Node.js

1.A Descripción de la Solución

La solución consiste en un servidor capaz de servir un formulario HTML inicial, interceptar el envío de datos mediante el método POST, procesar dicha información en el servidor y

devolver una segunda vista donde las variables son reemplazadas por los datos reales ingresados por el usuario.

1.B Implementación de Código

Vista de Formulario (vista1.html)

```
66 <body>
67   <div class="tarjeta">
68     <h1>¿Cuál es tu animal favorito?</h1>
69
70     <form action="/procesar" method="POST">
71       <label for="animal" style="color: ■ #666; font-size: 14px;">Nombre del animal:</label>
72       <input type="text" id="animal" name="animal" placeholder="Ej: León, Tortuga..." required>
73       <button type="submit">Descubrir</button>
74     </form>
75   </div>
76 </body>
```

Vista de Resultado (vista2.html)

```
<body>
  <div class="tarjeta">
    <h1>¡Excelente elección!</h1>

    <p>El animal que ingresaste como tu favorito es:</p>

    <!-- Variable con el animal ingresado por el usuario. -->
    <span class="resultado-animal">{{animalFavorito}}</span>

    <!-- Al hacer clic, el navegador hace un GET a la ruta '/' y devuelve al formulario -->
    <a href="/" class="boton-volver">Volver a intentar</a>
  </div>
</body>
</html>
```

Lógica del Servidor (server.js)

```

1  const http = require('http');
2  const fs = require('fs');
3  const querystring = require('querystring');
4
5  const server = http.createServer((req, res) => {
6
7      // Carga del formulario
8      if (req.url === '/' && req.method === 'GET') {
9          fs.readFile('vista1.html', 'utf8', (err, data) => {
10             if (err) {
11                 res.writeHead(500);
12                 return res.end('Error interno del servidor al cargar vista 1');
13             }
14             res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
15             res.end(data);
16         });
17     }
18
19     // Intercepción de los datos del formulario
20     else if (req.url === '/procesar' && req.method === 'POST') {
21         let cuerpoPeticion = '';
22
23         // Recibe los datos del formulario por fragmentos y los junta
24         req.on('data', chunk => {
25             cuerpoPeticion += chunk.toString();
26         });
27
28         // Una vez que se termina de recibir toda la info:
29         req.on('end', () => {
30             // Transforma el string recibido a un objeto
31             const datosParseados = querystring.parse(cuerpoPeticion);
32             const nombreDelAnimal = datosParseados.animal;
33
34             // Imprime en la terminal lo que el formulario envió
35             // Si sale undefined o vacío, hubo un problema con el formulario
36             console.log("Dato interceptado del formulario:", nombreDelAnimal);

```

```

38 // Lee la segunda vista para prepararla antes de enviarla
39
40 fs.readFile('vista2.html', 'utf8', (err, data) => {
41   if (err) return res.end('Error');
42
43   // Reemplazala variable del HTML por el dato interceptado
44   const paginaActualizada = data.replaceAll('{{animalFavorito}}', nombreDelAnimal);
45
46   // Regresa la nueva página con el nombre del animal ya puesto
47
48   res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
49   res.end(paginaActualizada);
50 });
51 });
52 }
53
54 // Ruta por defecto (Fallback)
55 else {
56   res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
57   res.end('Página no encontrada');
58 }
59 });
60
61 // Iniciamos la escucha del servidor en el puerto 3000
62 server.listen(3006, () => {
63   console.log('Servidor activo. Por favor, abre http://localhost:3006 en tu navegador.');
```

1.C Resultados

Muestra de Consola :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

C:\Program Files\nodejs\node.exe --experimental-network-inspection .\server.js
Servidor activo. Por favor, abre http://localhost:3006 en tu navegador.
```

Muestra de Vista 1 (Formulario):

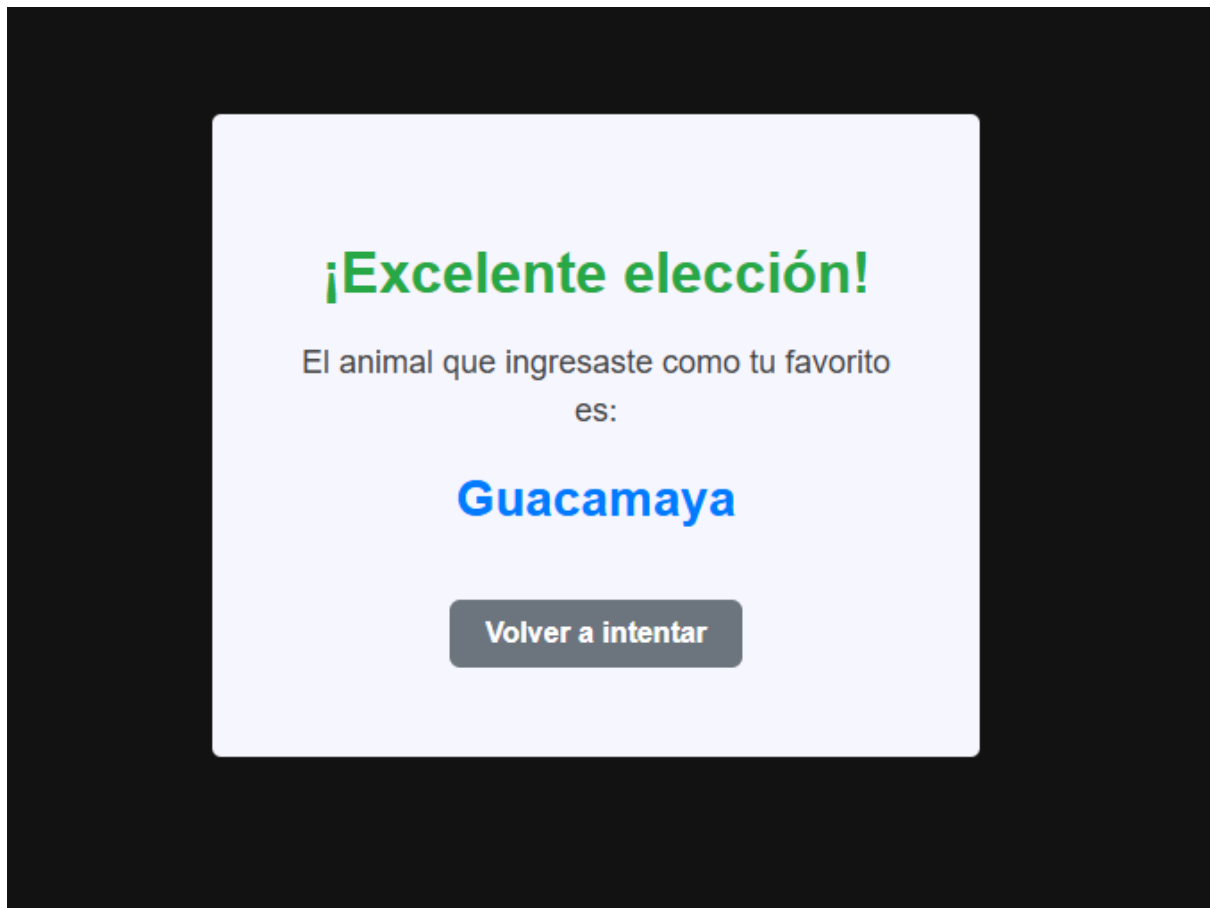
¿Cuál es tu animal favorito?

Nombre del animal:

Guacamaya

Descubrir

Muestra de Vista 2 (Resultado):



1.D Explicación de la Lógica

La lógica se basa en tres partes:

- **Escucha de Eventos:** Se utiliza `req.on('data')` para capturar los paquetes de datos que llegan desde el formulario, asegurando que se reciba la cadena completa antes de procesarla.
- **Intercepción:** Al detectar la ruta `/procesar`, el servidor detiene el flujo normal y extrae el valor asociado a la clave "animal".
- **Inyección de Datos:** Mediante el método `replaceAll()`, se modifica el contenido del archivo HTML en memoria antes de ser enviado al cliente.

Ejercicio 2: Uso de Enumeradores en TypeScript

2.A Descripción de la Solución

Se implementa un script en TypeScript que define categorías fijas para películas. Se utiliza el tipado estricto para evitar errores de escritura y se automatiza la visualización de estas categorías en la consola del sistema.

2.B Implementación de Código

```
1 // Un 'enum' agrupa opciones lógicas. Le asignamos un valor de texto (string)
2 // a cada opción para que, al mostrarlo en pantalla, sea legible para el usuario.
3
4 enum GeneroPelicula {
5     Accion = "Acción",
6     Comedia = "Comedia",
7     Drama = "Drama",
8     Terror = "Terror",
9     CienciaFiccion = "Ciencia Ficción"
10 }
11
12 enum PaisPelicula {
13     EstadosUnidos = "Estados Unidos",
14     Espana = "España",
15     Mexico = "México",
16     Argentina = "Argentina",
17     CoreaDelSur = "Corea del Sur"
18 }
19
20
21 // Usa Object.values() para extraer únicamente los valores de texto de nuestros enumeradores
22 // y convertirlos en una lista para poder recorrerla item por item.
23
24 console.log("GÉNEROS DE PELÍCULAS DISPONIBLES:");
25
26 // Recorre el array de géneros usando un forEach.
27 // Por cada iteración, 'genero' tomará el valor de la opción actual y la imprime.
28 Object.values(GeneroPelicula).forEach((genero) => {
29     console.log(`- ${genero}`);
30 });
31
32 // Se usa un \n para separar ambas listas en la consola.
33 console.log("\n PAÍSES DE ORIGEN DISPONIBLES:");
34
35 // Se usa la misma lógica de extracción e iteración para los países.
36 Object.values(PaisPelicula).forEach((pais) => {
37     console.log(`- ${pais}`);
38 });
```

2.C Resultado:

```
C:\Users\Jhairt06\Desktop\typescript>node ejercicio2.js
GÉNEROS DE PELÍCULAS DISPONIBLES:
- Acción
- Comedia
- Drama
- Terror
- Ciencia Ficción

PAÍSES DE ORIGEN DISPONIBLES:
- Estados Unidos
- España
- México
- Argentina
- Corea del Sur
```

2.D Explicación de la Lógica

El uso de enum en TypeScript proporciona una capa de abstracción donde el programador trabaja con nombres lógicos (ej. `Accion`) mientras que el sistema maneja valores legibles (ej. "Acción"). La integración con `Object.values()` permite transformar estas estructuras estáticas en arreglos dinámicos que pueden ser recorridos con ciclos, optimizando la visualización de datos sin necesidad de escribir manualmente cada línea de salida.

Referencias

Node.js Foundation. (2024). *HTTP module documentation*. <https://nodejs.org/api/http.html>

Microsoft Corporation. (2024). *TypeScript Handbook: Enums*.
<https://www.typescriptlang.org/docs/handbook/enums.html>

freeCodeCamp. (2024). *Aprende TypeScript - Curso desde cero*
<https://www.freecodecamp.org/espanol/news/aprende-typescript-curso-desde-cero>

W3Schools (2026). *Node.js Tutorial*
<https://www.w3schools.com/nodejs>

Introducción

El presente documento detalla la implementación técnica de dos ejercicios fundamentales en el desarrollo backend y tipado fuerte. El primer ejercicio aborda la creación de un servidor web utilizando las herramientas nativas de Node.js, prescindiendo de frameworks externos para comprender el ciclo de vida de una petición HTTP (Request) y la generación de una respuesta dinámica (Response) mediante la manipulación de flujos de datos.

El segundo ejercicio se centra en las ventajas de TypeScript para la definición de estructuras constantes y legibles mediante el uso de enumeradores (Enums). Esta práctica permite gestionar listas cerradas de datos, como géneros cinematográficos y países, garantizando la integridad de la información y facilitando el mantenimiento del código fuente.

1. Ejercicio 1: Intercepción de Peticiones y Manejo de Vistas en Node.js

1.A Descripción de la Solución

La solución consiste en un servidor capaz de servir un formulario HTML inicial, interceptar el envío de datos mediante el método POST, procesar dicha información en el servidor y devolver una segunda vista donde las variables son reemplazadas por los datos reales ingresados por el usuario.

1.B Implementación de Código

Vista de Formulario (vista1.html)

```
66 <body>
67   <div class="tarjeta">
68     <h1>¿Cuál es tu animal favorito?</h1>
69
70     <form action="/procesar" method="POST">
71       <label for="animal" style="color: #666; font-size: 14px;">Nombre del animal:</label>
72       <input type="text" id="animal" name="animal" placeholder="Ej: León, Tortuga..." required>
73       <button type="submit">Descubrir</button>
74     </form>
75   </div>
76 </body>
```

Vista de Resultado (vista2.html)

```
<body>
  <div class="tarjeta">
    <h1>¡Excelente elección!</h1>

    <p>El animal que ingresaste como tu favorito es:</p>

    <!-- Variable con el animal ingresado por el usuario. -->
    <span class="resultado-animal">{{animalFavorito}}</span>

    <!-- Al hacer clic, el navegador hace un GET a la ruta '/' y devuelve al formulario -->
    <a href="/" class="boton-volver">Volver a intentar</a>
  </div>
</body>
</html>
```

Lógica del Servidor (server.js)

```

1  const http = require('http');
2  const fs = require('fs');
3  const querystring = require('querystring');
4
5  const server = http.createServer((req, res) => {
6
7      // Carga del formulario
8      if (req.url === '/' && req.method === 'GET') {
9          fs.readFile('vista1.html', 'utf8', (err, data) => {
10              if (err) {
11                  res.writeHead(500);
12                  return res.end('Error interno del servidor al cargar vista 1');
13              }
14              res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
15              res.end(data);
16          });
17      }
18
19      // Intercepción de los datos del formulario
20      else if (req.url === '/procesar' && req.method === 'POST') {
21          let cuerpoPeticion = '';
22
23          // Recibe los datos del formulario por fragmentos y los junta
24          req.on('data', chunk => {
25              cuerpoPeticion += chunk.toString();
26          });
27
28          // Una vez que se termina de recibir toda la info:
29          req.on('end', () => {
30              // Transforma el string recibido a un objeto
31              const datosParseados = querystring.parse(cuerpoPeticion);
32              const nombreDelAnimal = datosParseados.animal;
33
34              // Imprime en la terminal lo que el formulario envió
35              // Si sale undefined o vacío, hubo un problema con el formulario
36              console.log("Dato interceptado del formulario:", nombreDelAnimal);

```

```

38 // Lee la segunda vista para prepararla antes de enviarla
39
40 fs.readFile('vista2.html', 'utf8', (err, data) => {
41   if (err) return res.end('Error');
42
43   // Reemplazala variable del HTML por el dato interceptado
44   const paginaActualizada = data.replaceAll('{{animalFavorito}}', nombreDelAnimal);
45
46   // Regresa la nueva página con el nombre del animal ya puesto
47
48   res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
49   res.end(paginaActualizada);
50 });
51 });
52 }
53
54 // Ruta por defecto (Fallback)
55 else {
56   res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
57   res.end('Página no encontrada');
58 }
59 });
60
61 // Iniciamos la escucha del servidor en el puerto 3000
62 server.listen(3006, () => {
63   console.log('Servidor activo. Por favor, abre http://localhost:3006 en tu navegador.');
```

1.C Resultados

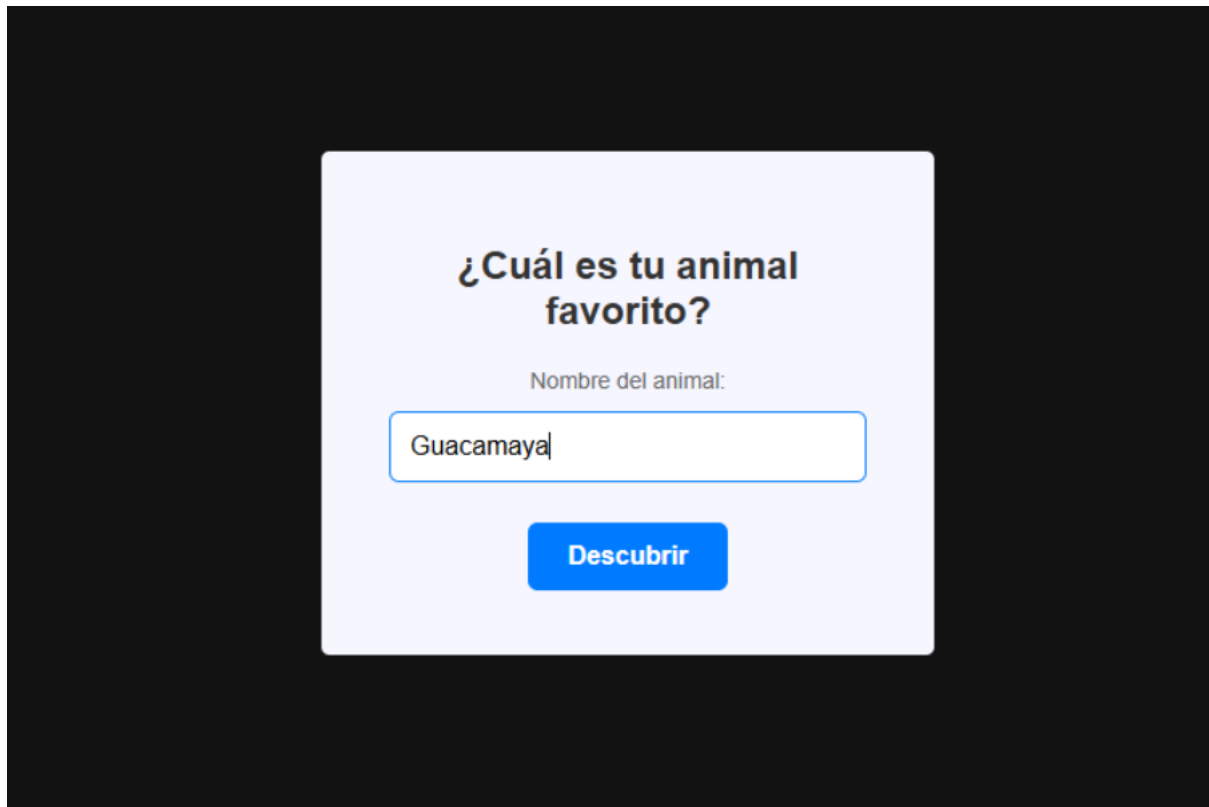
Muestra de Consola :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

C:\Program Files\nodejs\node.exe --experimental-network-inspection .\server.js
Servidor activo. Por favor, abre http://localhost:3006 en tu navegador.
```

Muestra de Vista 1 (Formulario):



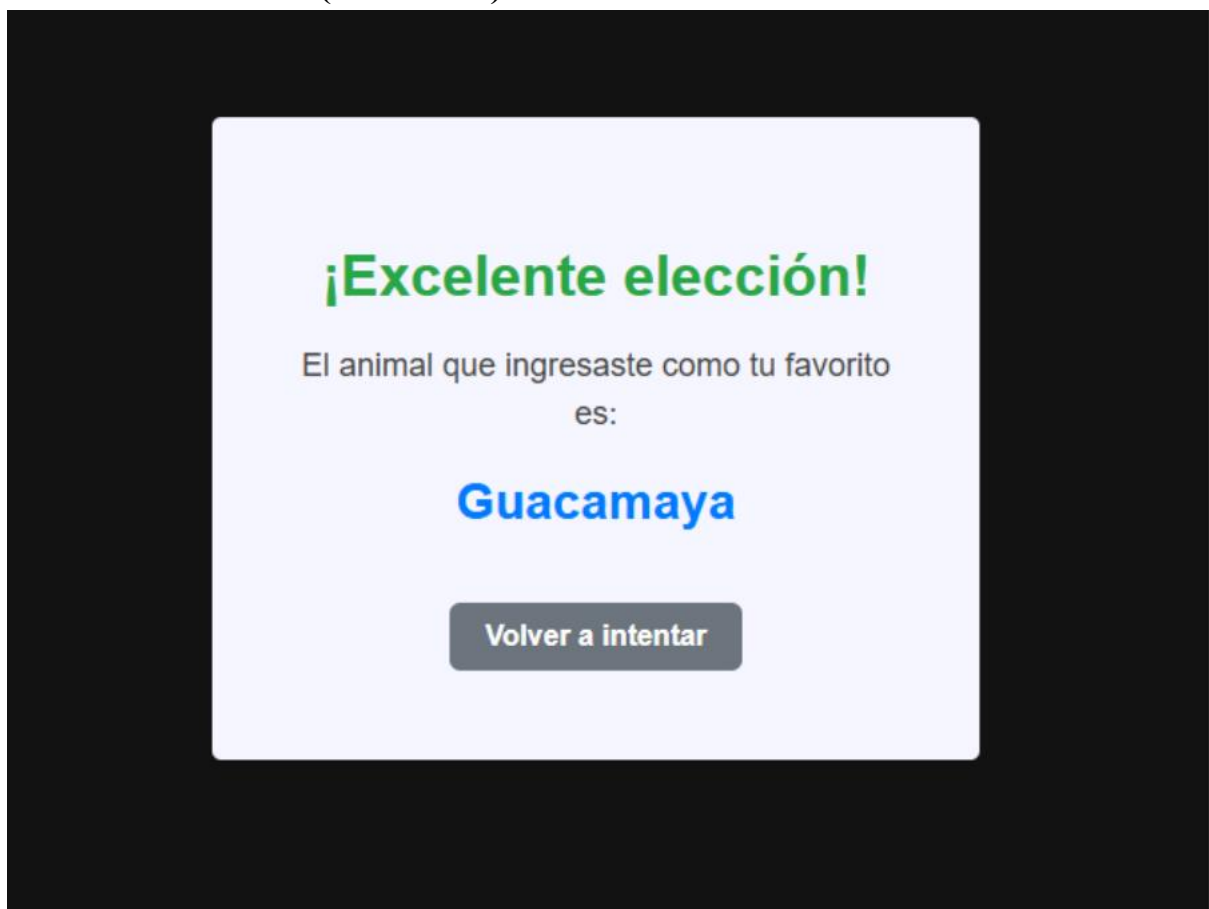
¿Cuál es tu animal favorito?

Nombre del animal:

Guacamaya

Descubrir

Muestra de Vista 2 (Resultado):



¡Excelente elección!

El animal que ingresaste como tu favorito es:

Guacamaya

Volver a intentar

1.D Explicación de la Lógica

La lógica se basa en tres partes:

- **Escucha de Eventos:** Se utiliza `req.on('data')` para capturar los paquetes de datos que llegan desde el formulario, asegurando que se reciba la cadena completa antes de procesarla.
- **Intercepción:** Al detectar la ruta `/procesar`, el servidor detiene el flujo normal y extrae el valor asociado a la clave "animal".
- **Inyección de Datos:** Mediante el método `replaceAll()`, se modifica el contenido del archivo HTML en memoria antes de ser enviado al cliente.

Ejercicio 2: Uso de Enumeradores en TypeScript

2.A Descripción de la Solución

Se implementa un script en TypeScript que define categorías fijas para películas. Se utiliza el tipado estricto para evitar errores de escritura y se automatiza la visualización de estas categorías en la consola del sistema.

2.B Implementación de Código

```
1 // Un 'enum' agrupa opciones lógicas. Le asignamos un valor de texto (string)
2 // a cada opción para que, al mostrarlo en pantalla, sea legible para el usuario.
3
4 enum GeneroPelicula {
5     Accion = "Acción",
6     Comedia = "Comedia",
7     Drama = "Drama",
8     Terror = "Terror",
9     CienciaFiccion = "Ciencia Ficción"
10 }
11
12 enum PaisPelicula {
13     EstadosUnidos = "Estados Unidos",
14     Espana = "España",
15     Mexico = "México",
16     Argentina = "Argentina",
17     CoreaDelSur = "Corea del Sur"
18 }
19
20
21 // Usa Object.values() para extraer únicamente los valores de texto de nuestros enumeradores
22 // y convertirlos en una lista para poder recorrerla item por item.
23
24 console.log("GÉNEROS DE PELÍCULAS DISPONIBLES:");
25
26 // Recorre el array de géneros usando un forEach.
27 // Por cada iteración, 'genero' tomará el valor de la opción actual y la imprime.
28 Object.values(GeneroPelicula).forEach((genero) => {
29     console.log(`- ${genero}`);
30 });
31
32 // Se usa un \n para separar ambas listas en la consola.
33 console.log("\n PAÍSES DE ORIGEN DISPONIBLES:");
34
35 // Se usa la misma lógica de extracción e iteración para los países.
36 Object.values(PaisPelicula).forEach((pais) => {
37     console.log(`- ${pais}`);
38 });
```

2.C Resultado:

```
C:\Users\Jhairt06\Desktop\typescript>node ejercicio2.js
GÉNEROS DE PELÍCULAS DISPONIBLES:
- Acción
- Comedia
- Drama
- Terror
- Ciencia Ficción

PAÍSES DE ORIGEN DISPONIBLES:
- Estados Unidos
- España
- México
- Argentina
- Corea del Sur
```

2.D Explicación de la Lógica

El uso de enum en TypeScript proporciona una capa de abstracción donde el programador trabaja con nombres lógicos (ej. `Accion`) mientras que el sistema maneja valores legibles (ej. "Acción"). La integración con `Object.values()` permite transformar estas estructuras estáticas en arreglos dinámicos que pueden ser recorridos con ciclos, optimizando la visualización de datos sin necesidad de escribir manualmente cada línea de salida.

Referencias

Node.js Foundation. (2024). *HTTP module documentation*. <https://nodejs.org/api/http.html>

Microsoft Corporation. (2024). *TypeScript HandBook: Enums*.
<https://www.typescriptlang.org/docs/handbook/enums.html>

freeCodeCamp. (2024). *Aprende TypeScript - Curso desde cero*

<https://www.freecodecamp.org/espanol/news/aprende-typescript-curso-desde-cero>

W3Schools (2026). *Node.js Tutorial*

<https://www.w3schools.com/nodejs>